Marathos lab

Purpose of this project is to use your knowledge of Databricks and data engineering to build data platform and pipeline for business stakeholders to aid in data driven decisions. You are working for a global company called Marathos, which hosts marathons all over the world.

Image generated by Nano Banana 2.

Background

Marathos is a traditional company that have been hosting marathons during very long time. They have started their digital and data journey, which they plan to expand. They have through LinkedIn seen some students in Yrkeshögskola post cool data engineering projects, both within data platforms and data visualization. Now they want to have a collaboration project with your school, to showcase a project within their particular tech stack. The tech stack for Marathos are:

- databricks
- python
- pyspark
- plotly
- git and github
- more but not relevant for this project

Their current data engineer have specified some tasks that they want you to complete for this project.

Task 0 - onboarding

a) Go through 04-10 in this github repository to learn the very basics of medallion architecture and databricks.

b) Also create a github repository and link it to your databricks account so that you can commit and push code from databricks to that repo. If you have forgotten how to do this - follow 01_databricks_setup. Do this even if you have a code alongs repo as this is a good practice to setup from scratch again.

c) Create one ETL pipeline in your repo that will have the following folder structure

```
marathos_<fname>_<lname>
├── dimensional_modeling
├── explorations
├── transformations
│   ├── bronze
│   ├── gold
│   └── silver
└── utils
```

d) Now create an SQL script or a notebook to build up the unity catalog to look like this.

```
marathos
├── bronze
├── default
│   └── raw
├── gold
├── information_schema
└── silver
```

Now upload this dataset into your unity catalog. Note that you may choose to have more directories inside of the volume if you need.

Task 1 - EDA

Create an EDA of the data. Here are some simple questions to follow as a starting point

- find out number of rows
- find out number of columns
- check schema if you have inferred it or specify an appropriate schema
- descriptive summary of numerical fields
- count nulls for each field
- how many unique events are their?
- how are ages distributed among the runners?
- which countries are most represented?

Task 2 - Bronze layer

Ingest the data into the bronze layer.

Task 3 - Silver layer

Clean the data and output a one big table (obt) of the cleaned data. Here are some starting point to investigate

- check validity status of event and performance
    - if event has unit km or mi then performance should be in h
    - if event has unit h then performance should be in km

    - remove all invalid rows (for simplicity you can consider d (days) as invalid and remove it)
- convert time of athelete performance to appropriate data type

Create relevant ids, which will be important later on dimensional modeling.

For the ids for example event_id you can use the function `dense_rank()` to produce the id based on event_name. Here is an example code snippet

```
from pyspark.sql.functions import dense_rank
from pyspark.sql.window import Window

w = Window.orderBy("Event name")
df.withColumn("event_id", dense_rank().over(w)).select(
    "Event name", "event_id"
).display()
```

Other cleaning of your choice ...

Task 4 - Dimensional modeling

The data engineer has provided you with a starting dimensional model that you should continue developing.

Here is a starting dbml code for the dimensional model that you can directly paste into dbdiagram.

```
Table fct_results {
  result_id integer
  event_id integer
  performance float
  // more columns
}

Table dim_event {
  event_id integer [primary key, ref:< fct_results.event_id]
  event_type string
  // more columns
}

Table dim_athlete {
  athlete_id integer
  // more columns
}
```

Task 5 - Gold layer

Build your gold layer based on your dimensional model. For this layer you should also create at least two views for each of the marathon types (distance or length). Feel free to create more views.

Task 6 - Dashboard

Build a user friendly dashboard that shows a few metrics, a few graphs and a link to genie chatbot, see task 7.

Task 7 - Marathos Genie

Business stakeholders really enjoy to ask questions to data analyst about the data and the data analysts usually perform ad hoc SQL queries on the data to answer their questions. In this task you will create a Genie for Marathos, where stakeholders can ask questions to instead of disturbing the data analysts with these simple ad hoc questions. Make sure to link appropriate datasets to Marathos Genie.

Before we can give this to the stakeholders to play, it's very important for the data engineer that the answers are reliable or else the data team is screwed. Write a few questions or click on a few questions recommended questions and then do manual calculations in a notebook to verify the correctness of the outputs.

BONUS

- country data - either use LLM to simulate good guess of abbreviations or find a real one. Ingest this data in and use it to enhance the downstream layers.
- use LLM to create a new marathon with data and place it in to unity catalog and stream it
- schedule the pipeline
- dashboard is comprehensive and have good insights
- correct terminology and language when presenting in the video
- create a date dimension - here you can use the starting date, but note that the data is very tedious

Task 8 - Video

Create a 5-10 min long video where you showcase your dashboard, describe your data pipeline and shortly your code structure. Also make sure to demo the things you have done, for example Genie, ETL pipeline, and also other things that you have done in BONUS you should also demo them.

Submission

Hand in the following to your learning platform:

- a link to your public github repository for this lab
- a link to your video recording (for example you could use unlisted youtube video)

Grading

If you have taken ideas of codes from someone or used LLM, it is **important** that you state the source and understand how the codes work. Write a comment next to these codes. LLM is okay for smaller tasks, asking for ideas, concepts but not for solving entire tasks.

Criteria for G:

- several relevant commits
- video is clear and shows you understand what you have developed

- solved all tasks excluding BONUS

Criteria for VG:

- implemented the BONUS parts
- video is clear and holds high technical quality with correct terminologies
- code is well structured following DRY principle
- repo is well structured